

PART 1 — Git & GitHub

Version Control · Branching · Merge/Rebase · Stash · Pull Requests

1.1 What is Git?

Git = Version Control System — tracks code changes, enables rollback & team collaboration. Saves history of your code *locally* on your computer.

GitHub = Website that stores Git repos *online* & enables teamwork, code review, pull requests.

Core Idea	What it does
git add + git commit	Save a snapshot of your work
git branch + git merge	Work on features separately without breaking anything
git push + git pull	Sync your code with GitHub

1.2 Setup & Configuration

Command	Purpose
git config --global user.name "Your Name"	Set global username
git config --global user.email "you@email.com"	Set global email
git init	Initialize a new local repository
git clone <url>	Clone a repo from GitHub to local
git remote add origin <link>	Connect local repo to GitHub remote
git remote -v	Verify connected remote URLs

1.3 Workflows

First-Time Push to GitHub — run these 5 steps once:

```
git init → git add . → git commit -m 'msg' → git remote add origin <url> → git push -u origin main
```

After First Push — just 3 steps every time:

```
git add . → git commit -m 'Update' → git push
```

1.4 Core Commands Reference

Category	Command	What it Does
Status	git status	See what files have changed
Status	git log --oneline	See commit history (compact view)
Staging	git add filename.txt	Stage a specific file
Staging	git add .	Stage ALL changed files
Commit	git commit -m "My change"	Save a snapshot with a message
Push	git push -u origin main	First-time push (sets upstream)

Category	Command	What it Does
Push	git push	Push after the first time
Pull	git pull	Fetch + merge latest code from GitHub
Fetch	git fetch	Download changes — does NOT merge yet

1.5 Undoing Things

Command	What it Undoes
git restore --staged file	Unstage a file (keep changes in working dir)
git restore filename.txt	Discard local file changes (permanent!)
git revert <commit-id>	Create new commit that reverses a commit (safe)
git reset --soft HEAD~1	Undo last commit — keep changes STAGED
git reset --mixed HEAD~1	Undo last commit — changes go to working dir
git reset --hard HEAD~1	■ Undo last commit AND discard all changes (DANGEROUS)
git push --force origin main	■ Overwrite remote branch (WARNING: destroys history)

■ **Danger:** git reset --hard permanently discards changes with no undo. git push --force overwrites the remote branch — can erase teammates' work. Prefer git revert for safe rollback.

1.6 Git Stash

git stash = Temporarily save uncommitted changes (staged + unstaged) to get a clean working directory. Use when you want to switch branches but don't want to commit yet.

Command	Action
git stash	Stash current uncommitted changes
git stash pop	Apply stash AND remove it from stash list
git stash apply	Apply stash but KEEP it in stash list
git stash list	Show all saved stashes
git stash drop	Delete a specific stash entry

1.7 Branching

Branch = A separate copy of your code where you can make changes safely without touching *main*. Branch off main → develop feature → merge back.

Command	Action
git branch	List all branches
git branch feature-name	Create a new branch
git checkout feature-name	Switch to existing branch
git checkout -b feature-name	Create AND switch (shortcut)

Command	Action
git merge feature-name	Merge feature into current branch
git branch -d name	Delete branch (safe — won't delete if unmerged)
git branch -D name	Force delete a branch

1.8 Merge vs Rebase

git merge	git rebase
Keeps full branch history	Rewrites history — linear path
Adds a merge commit	No merge commit created
Preserves exactly what happened	Rewrites commit IDs
Safe — doesn't alter existing commits	DANGEROUS if branch is already shared
Resolves all conflicts at once	Resolves conflicts commit-by-commit

■ **Tip:** Use merge for shared/public branches. Use rebase only on your own local branches to clean up before merging. NEVER rebase a branch others have already pulled.

1.9 Merge Conflicts & Pull Requests

Conflict = Two people edited the same line differently — Git can't auto-merge.

Conflict Resolution Steps	
1. Open affected files	Git marks conflict areas with <<<< ==== >>>>
2. Manually pick changes to keep	Edit the file to show the final desired state
3. Remove conflict markers	Delete all <<<< ==== >>>> lines
4. Stage & commit	git add → git commit

Pull Request (PR) = Request to merge your branch into main. Team reviews, comments, then approves or requests changes.

Step	Action
1	Push your branch to GitHub
2	Open PR on GitHub
3	Team reviews & comments
4	Approve / Request Changes
5	Merge into main

1.10 Git Interview Q&A;

Question	Answer
What is version control?	System that manages & tracks code changes so we can go back to earlier versions

Question	Answer
What is .gitignore?	File telling Git which files/folders NOT to track (node_modules, .env, build files)
What is git HEAD?	Special reference indicating your current location in repository history
What is a commit?	Saved snapshot of your project files at a specific point in time
git merge vs git rebase?	Merge preserves history + adds merge commit; Rebase rewrites history for linear path
git pull vs git fetch?	git pull = fetch + auto merge; git fetch = download only, you review before merging
What is git stash?	Temporarily saves uncommitted changes so you can switch branches cleanly
What is a branch?	Separate copy of code for safe feature development without touching main

PART 2 — REST API

Architecture · HTTP Methods · Status Codes · Auth/JWT · SOAP

2.1 What is an API?

API = Application Programming Interface — set of rules allowing software systems to communicate & exchange data.
Used for: Frontend ↔ Backend, Backend ↔ Database, Backend ↔ Payment Gateway.

2.2 What is REST API?

REST = Representational State Transfer — architectural rules for building APIs. Uses HTTP Protocol, URIs, JSON data to communicate between client & server.

Rule	REST Rule	Explanation	Example
1	Use URLs as Nouns	URL identifies the resource — not the action	/users not /getUsers
2	HTTP Method = Verb	GET to read, POST to create, PUT/PATCH to update, DELETE to remove	GET /users/42
3	Respond with JSON	Data sent back as structured JSON — both apps understand	{"id":1,"name":"Priya"}
4	Stateless	Every request is independent — server does NOT remember previous requests	Each call has all needed info

2.3 HTTP Methods (CRUD)

Method	CRUD	Idempotent?	Example Endpoint	Use Case
GET	Read	✓ Yes	GET /users	Fetch all users
GET	Read	✓ Yes	GET /users/42	Get user with ID 42
POST	Create	✗ No	POST /users	Create a new user (new each time)
PUT	Update	✓ Yes	PUT /users/42	Replace user 42 completely
PATCH	Update	~ Dep	PATCH /users/42	Update only user 42's name
DELETE	Delete	✓ Yes	DELETE /users/42	Delete user 42

2.4 Request & Response Structure

REQUEST contains	RESPONSE contains
Method + URI + HTTP Version e.g. GET /users/42 HTTP/1.1	Status Code + Status Message e.g. HTTP/1.1 200 OK
Headers → extra info (Host, Authorization, Content-Type: application/json)	Headers → metadata about response (Content-Type, Cache-Control)
Body → data you're sending (POST/PUT only) { "name": "Priya", "email": "p@x.com" }	Body → actual data returned (usually JSON) {"id":42, "name":"Priya", "email":"p@x.com"}

2.5 HTTP Status Codes

Code	Name	Meaning
200	OK	GET worked, data returned
201	Created	POST worked, new item created
204	No Content	DELETE worked, nothing to return
400	Bad Request	Invalid data sent by client
401	Unauthorized	Not logged in / token missing or invalid
403	Forbidden	Logged in but no permission to access
404	Not Found	Wrong URL — resource doesn't exist
500	Server Error	Bug on the server side

■ **Tip:** 2xx = Success | 4xx = Client Error (your problem) | 5xx = Server Error (their problem)

2.6 Rate Limiting, Versioning & Idempotency

Concept	Description
Rate Limiting	Max requests a user can make in a time period — prevents overload, ensures fairness
API Versioning	Change API without breaking existing users — put version in URL (/v1/users, /v2/users)
Idempotency	Sending same request multiple times gives same result (GET, DELETE, PUT = idempotent; POST = not)

Concept	Idempotent?	Reason
GET	✓ Yes	Just reads data — same result every time
DELETE	✓ Yes	Delete once → deleted. Delete again → still deleted. Same final state.
PUT	✓ Yes	Replacing all data with same data = same result
POST	✗ No	Creates NEW resource each time — different result every call
PATCH	~ Depends	Depends on implementation

2.7 Authentication vs Authorization & JWT

Authentication	Authorization
Verifying WHO the user IS	Checking WHAT user can access
Login with username/password	Checks user's roles & permissions
Token issued after successful login	Happens AFTER authentication
Answers: Are you who you claim?	Answers: Are you allowed to do this?

Concept	Definition
JWT (JSON Web Token)	Compact token for auth — structure: Header.Payload.Signature (Base64 encoded)
Stateless	Server does NOT store session — every request must carry all required info (e.g. JWT in header)

Concept	Definition
CORS	Cross-Origin Resource Sharing — frontend from one origin accessing backend from another origin

2.8 Synchronous vs Asynchronous API

Synchronous API	Asynchronous API
Client waits for response before continuing	Client does NOT wait — continues other tasks
Blocking operation	Non-blocking operation
Simple to implement	Better for long-running operations
Good for real-time operations	Good: file processing, emails, background jobs

REST API vs SOAP

REST API	SOAP
Simple, flexible architecture	Protocol-based (strict rules)
JSON or XML data format	XML only — verbose
Fast & lightweight	Slower but enterprise-grade
HTTPS + OAuth security	WS-Security
Salesforce preferred	Still supported in Salesforce

PART 3 — Salesforce Platform

Objects • Apex • Triggers • LWC • Flows • Integration • Security • Deployment

S01 Cloud Computing

Model	Full Name	User Manages	Provider Manages	Example
IaaS	Infrastructure as a Service	OS, Apps, Data	Hardware, Network, Storage	AWS EC2, Azure VM
PaaS	Platform as a Service	Application & Data	OS, Runtime, Infrastructure	Salesforce, Google App Engine
SaaS	Software as a Service	Only uses the app	Everything	Gmail, Salesforce CRM
On-Premise	Self-hosted	EVERYTHING — full setup	N/A	Company's own server

S02 MVC Architecture

Layer	What It Is	In Salesforce
Model (Data)	Represents data & data structure	Objects, Fields, Records (Standard & Custom)
View (UI)	Represents what user sees	Standard UI, Visualforce Pages, LWC, Reports & Dashboards
Controller (Logic)	Handles user action & business logic	Standard Controller, Custom Controller, Controller Extension, Apex, Flows

S03 Salesforce Org Types

Org Type	Connected to Prod?	Real Data?	Use Case
Developer Edition	✗ No	✗ No	Free, learning, individual development
Developer Sandbox	✓ Yes	✗ No	Individual development, testing small features
Developer Pro Sandbox	✓ Yes	✗ No	Development with larger datasets, better testing
Partial Copy Sandbox	✓ Yes	Partial sample	Testing with real data samples, UAT
Full Sandbox	✓ Yes	Complete copy	Final deployment testing, full load testing
Production Org	N/A	LIVE data	Day-to-day business operations

S04 Objects & Fields

Standard Objects	Custom Objects
Pre-defined by Salesforce	Created by user/developer
Cannot be deleted	Can be deleted
Customization is limited	Fully customizable
Normal naming (Account, Contact, Lead)	Name ends with __c (Student__c, Course__c)

Field Type	Description	Example
Text	Plain text up to 255 chars	First Name, City
Number	Numeric values (integer)	Quantity, Score
Decimal / Currency	Numeric with decimal places	Price, Salary
Picklist	Predefined dropdown list of options	Status: Active / Inactive / Pending
Date	Calendar date (no time)	Birth Date, Start Date
DateTime	Date + Time combined	Meeting Scheduled At
Checkbox	True / False boolean	Is Active?, Is Verified?
Lookup	Weak relationship link to another object	Account on Contact (optional)
Master-Detail	Strong relationship — parent-child link	OrderItem on Order (mandatory)
Formula	Calculated read-only field based on formula	Total = Qty * Price
Roll-Up Summary	Aggregate value from child records (MD only)	Total Orders COUNT on Account
Auto Number	Auto-incremented unique number	CASE-0001, CASE-0002

S05 Relationships in Salesforce

Feature	Master-Detail	Lookup
Coupling	Strong — tightly coupled	Weak — loosely coupled
Child exists without parent?	✗ No — mandatory	✓ Yes — optional
Delete parent → child?	Cascade deletes child automatically	Does NOT delete child
Roll-up Summary Fields	✓ Supported (COUNT, SUM, MIN, MAX)	✗ Not supported
Child inherits security	✓ Yes — inherits OWD sharing settings	✗ No inheritance
Max per object	2 Master-Detail relationships	25 Lookup relationships
Example	Order → Order Items	Contact → Account

■ **Tip:** Many-to-Many = Junction Object with TWO Master-Detail relationships — e.g. Student__c & Course__c via Enrollment__c

S06 Salesforce Security Model

Security Layer	Controls	How Configured
Organization Level	Login hours, IP restrictions, password policies	Company-wide settings in Setup
Object Level (OLS)	Which objects a user can see/create/edit/delete (CRUD)	Profiles & Permission Sets
Record Level (Sharing)	Which specific records a user can access	OWD, Sharing Rules, Role Hierarchy, Manual Sharing
Field Level Security	Which fields a user can see or edit per object	Profile → Field Accessibility settings

Concept	Definition
Profile	Every user must have exactly ONE profile — controls baseline access (CRUD on objects, FLS, login hours)
Permission Set	Grants ADDITIONAL access on top of profile — additive only, cannot restrict. User can have multiple.
OWD	Org-Wide Default — default access level for all records: Private Read Only Public Read/Write
Role Hierarchy	Tree structure — users higher up can see records owned by users below them

S07 Tabs & Apps

Tabs = UI navigation elements giving access to objects, records & apps.

Apps = Collection of related Tabs, Objects & Features grouped for a specific business purpose.

Step	Action to Create a Lightning App
1	Setup → App Manager → New Lightning App
2	Name & Description
3	Navigation Style
4	Add Items & Utility Bar
5	Assign User Profile
6	Save & Finish

S08 SOQL & SOSL

SOQL (Salesforce Object Query Language) = Retrieve data from Salesforce objects — similar to SQL but for Salesforce

```
SELECT Name, LastModifiedDate FROM Account WHERE Name LIKE 'xyz%' ORDER BY LastModifiedDate
DESC LIMIT 5
-- Child→Parent: SELECT Teacher__r.Name FROM Student__c WHERE Name = 'Arjun'
-- Aggregate: SELECT Teacher__r.Name, COUNT(Id) FROM Student__c GROUP BY Teacher__r.Name
```

SOSL (Salesforce Object Search Language) = Search text across multiple objects & fields at once

```
FIND 'Arjun' IN NAME FIELDS RETURNING Account(Name), Contact(FirstName, LastName)
```

Feature	SOQL	SOSL
Purpose	Retrieve specific records from known object	Search text across multiple objects/fields
Objects at once	One object at a time (with relationships)	Multiple objects simultaneously
Query type	Structured query — like SQL WHERE clauses	Full-text search
When to use	You know which object — need specific records	Unknown location — search everywhere
Speed	Optimized for structured data retrieval	Faster for full-text/keyword searching

S09 DML Operations

DML = Data Manipulation Language — used to insert, update, delete & restore records in Apex

Operation	Apex Code	Description
INSERT	Account acc = new Account(Name='ABC'); insert acc;	Create a new record
UPDATE	acc.Name = 'xyz pvt'; update acc;	Update existing record
DELETE	delete acc;	Delete a record (goes to Recycle Bin)
UPSERT	upsert acc;	Insert if not exists, Update if already exists
UNDELETE	undelete acc;	Restore deleted record from Recycle Bin

S10 Formula Fields, Roll-Up & Validation Rules

Feature	Formula Field	Normal Field
Storage	Auto-calculated — no manual update	Stored in database directly
Accuracy	Always consistent & accurate	User manually enters/updates
Editability	Read-only — cannot edit	Can be edited freely
Automation	Cannot trigger workflows	Can trigger workflows & automations

Feature	Details
Formula Field	Calculated in real-time Value NOT stored in DB Always accurate Cannot be edited directly
Roll-Up Summary	Only works with Master-Detail Max 25 per object Supports filter criteria on child records
Validation Rules	Ensure data meets conditions BEFORE saving — If formula=True → Error & not saved

S11 Flows & Approval Process

Flow Type	Triggered By	Use Case Example
Screen Flow	User Interface (manual)	Forms, guided processes — Student registration form

Flow Type	Triggered By	Use Case Example
Record-Triggered Flow	Record created/updated/deleted	Auto-update related records — Update status when opportunity stage changes
Scheduled-Triggered Flow	Specific time or interval	Batch processing — Send daily reminder email at 9am
Autolaunched Flow	Called by process/API/code	Background automation logic — Data processing, integration tasks

Approval Process = Automated routing of records for approval — can be Approved, Rejected, or Recalled

Step	Stage
1	Employee Submits
2	Manager Receives notification
3	Manager Reviews
4	Approve / Reject
5	Employee Notified of outcome

S12 Visualforce vs LWC

Visualforce	Lightning Web Component (LWC)
Server-side rendering (slower)	Client-side rendering (faster)
Apex + HTML custom tags	Standard HTML5, CSS, JavaScript
Introduced 2008	Introduced 2019
Mobile support limited	Full mobile support
Less reusable	Highly reusable components
Older style, harder to test	Modern, easier testing with Jest

LWC File	Purpose
ComponentName.html	UI markup — defines structure using <template> tags
ComponentName.js	Business logic, event handling, data manipulation
ComponentName.js-meta.xml	Config: API version, target pages, isExposed
ComponentName.css	(optional) Component-specific CSS styling

S13 LWC Component Communication

Type	Direction	Mechanism	Example
Parent → Child	Data flows down	@api decorator on child property	<c-child message='Hello'> / Child: @api message;
Child → Parent	Events bubble up	CustomEvent + dispatchEvent()	Child: this.dispatchEvent(new CustomEvent('notify',{detail:msg}))
Sibling → Sibling	Via LMS	Lightning Message Service pub/sub	Publisher: publish(msg); Subscriber: subscribe(handleMsg)

S14 Visualforce Controllers

Controller Type	Description	Coding?
Standard Controller	Built-in Salesforce controller for standard/custom objects. Auto-provides view, edit, save, delete without any Apex code.	✗ No code needed
Custom Controller	Completely custom Apex class. Full control over data access, business logic, user action, CRUD operations.	✓ Apex class
Controller Extension	Extends Standard or Custom controller. Adds extra methods to existing controller without replacing it.	✓ Partial Apex

S15 Apex — Salesforce Programming Language

Apex = Strongly typed, OOP language by Salesforce for server-side logic. Java-like syntax, case-sensitive.

Data Type	Example	Notes
Integer	Integer i = 10;	Whole numbers only
String	String s = 'Hello';	Text values
Boolean	Boolean b = true;	True or False
Decimal	Decimal d = 3.14;	Decimal numbers with precision
Double	Double x = 1.23;	Floating point numbers
Date	Date d = Date.today();	Date without time
Datetime	Datetime dt = Datetime.now();	Date with time
Id	Id recId = '001xxxxx';	Salesforce 15/18 char record ID
Long	Long l = 123456789L;	Large whole numbers

Collection Type	Description	Example
List	Ordered, allows duplicates, indexed	List<String> colors = new List<String>();
Set	Unique values, unordered	Set<String> names = new Set<String>();
Map	Key-value pairs	Map<Integer,String> students = new Map<Integer,String>();
Enum	Fixed set of constants	Variables can only take one value from predefined list

■ **Tip:** Apex Transaction = Set of operations as one unit — if one fails, ALL fail (all-or-nothing default)

S16 Apex Triggers

Triggers = Apex code that executes automatically when records are inserted, updated, deleted or undeleted

Feature	Before Trigger	After Trigger
Execution Time	BEFORE saving to database	AFTER saving to database
Modify Trigger.new	✓ Yes — can modify fields	✗ No — read-only
DML to save changes	✗ Not needed — auto-saved	✓ Yes — must use separate DML
Access to Record ID	✗ Not available yet	✓ Available (record saved)

Feature	Before Trigger	After Trigger
Use Case	Validation, set defaults, format data	Update related records, send emails

Context Variable	Description	Available On
Trigger.new	List of new/updated records	Insert, Update, Undelete
Trigger.old	List of old records before change	Update, Delete
Trigger.isInsert	True if insert operation	All
Trigger.isUpdate	True if update operation	All
Trigger.isBefore	True if Before Trigger	All
Trigger.isAfter	True if After Trigger	All
Trigger.newMap	Map of Id → new records	Insert, Update, Undelete
Trigger.oldMap	Map of Id → old records	Update, Delete

■ **Danger:** TRIGGER GOLDEN RULES: (1) NEVER put SOQL inside a loop — causes SOQL limit exceeded (100 limit). (2) NEVER put DML inside a loop — causes DML limit exceeded (150 limit). (3) Use collections (List, Map) to bulk-collect data → query/update ONCE outside loop. (4) Always write bulkified triggers — Salesforce processes up to 200 records at once per batch.

S17 Governor Limits

Governor Limits = Limits enforced by Salesforce on Apex to control shared resource consumption. Purpose: Fair resource use | System stability | Efficiency | Prevent infinite loops

Limit Type	Synchronous	Asynchronous
SOQL queries per transaction	100	200
DML statements per transaction	150	150
Records retrieved by SOQL	50,000	50,000
Records via Database.getQueryLocator	10,000	50,000,000 (Batchable)
CPU time	10,000 ms (10 sec)	60,000 ms (60 sec)
Heap size	6 MB	12 MB
Callouts (HTTP requests)	100 per transaction	100 per transaction

S18 Asynchronous Apex

Async Apex = Runs in background separately from main transaction — higher governor limits, non-blocking

Type	Annotation / Interface	Use Case	Key Feature
Future Method	@future	Simple async tasks; callouts to external services	Runs after main transaction completes
Queueable Apex	implements Queueable	Advanced async; job chaining; monitor status	Can chain jobs, pass complex objects

Type	Annotation / Interface	Use Case	Key Feature
Batch Apex	implements Database.Batchable	Process millions of records in chunks	200 records/batch default, up to 50M records
Scheduled Apex	implements Schedulable	Run at scheduled time (daily/weekly)	Like a cron job — runs on schedule

S19 Database Class vs Standard DML

Database Class	Standard DML
Enhanced control & error handling	All-or-nothing by default
allOrNone=false → partial success allowed	If one record fails → ALL fail
Database.SaveResult → check which failed	Simpler syntax for straightforward ops
Better for bulk operations	Good for simple single-record operations

```
// Partial success – save what passes, skip what fails:
List results = Database.insert(recordList, false);
// false = allOrNone=false → allows partial success
```

S20 Salesforce Integration

Architecture	Description	Best For
Point-to-Point	Direct connection between each pair of systems. Simple but gets complex fast ($N*(N-1)/2$ connections).	Small: 2-3 systems
Hub & Spoke	Central hub connects all — all data passes through hub. Each system connects only to hub.	Medium: centralized control
Enterprise Service Bus	Middleware that routes, transforms, manages messages. Protocol conversion support.	Large enterprise

Direction	Description	Common Method
Inbound REST	External system → Salesforce. Salesforce receives & processes.	Standard REST API or Custom Apex @RestResource
Outbound REST	Salesforce → External system via Apex Callout.	HttpRequest/HttpResponse in Apex
Inbound SOAP	External system sends SOAP request to Salesforce WSDL.	Enterprise/Partner WSDL
Outbound Message	Salesforce auto-sends SOAP when record criteria met.	Workflow Outbound Message

```
// Outbound Apex Callout:
HttpRequest req = new HttpRequest();
req.setEndpoint('https://api.example.com/data');
req.setMethod('GET');
HttpResponse res = new Http().send(req);
System.debug(res.getBody());
```

Credential Method	Description
Named Credentials (Preferred)	Securely store endpoint + auth details — authentication handled automatically
Remote Site Settings	Whitelist external URLs — less secure, stores URL only, manual auth in Apex

S21 Profiles, Permission Sets & Sharing

Concept	Definition	Key Point
Profile	Controls baseline permissions — object CRUD, FLS, login hours, app access	Every user MUST have exactly ONE profile
Permission Set	Grants ADDITIONAL access on top of profile — additive only, cannot restrict	User can have multiple permission sets
Permission Set Group	Bundle multiple permission sets for easier assignment	Assign group instead of individual sets
Role	Defines user's position in hierarchy — determines record visibility	Higher role = sees records of users below
OWD	Default access level for records: Private Read Only Public Read/Write	Most restrictive setting for ALL users
Sharing Rules	Auto-share records with users/groups based on criteria	Opens access beyond OWD — additive only
Manual Sharing	Record owner manually shares one record with specific users	One-time, record-specific only

S22 Reports & Dashboards

Report Type	Description	Example
Tabular	Simple list of records with columns — like spreadsheet, no grouping	List of all Accounts
Summary	Groups records by field — shows subtotals per group	Opportunities grouped by Stage
Matrix	Groups by both rows AND columns — like a pivot table	Revenue by Region and Quarter
Joined	Multiple report blocks (different types) in one view	Accounts with Contacts and Cases combined

Concept	Details
Dashboard Components	Chart Gauge Metric Table Visualforce Page
Dynamic Dashboard	Shows data specific to the logged-in viewer (each user sees their own data)
Running User	The user whose security determines what data the dashboard shows

S23 Process Automation Comparison

Tool	Type	When to Use	Complexity
Screen Flow	Declarative	Guided user interaction, multi-step forms	Low — admin can build

Tool	Type	When to Use	Complexity
Record-Triggered Flow	Declarative	Auto-update records on create/update/delete	Low-Medium
Scheduled Flow	Declarative	Batch updates at specific scheduled times	Medium
Approval Process	Declarative	Multi-step approval routing with notifications	Low-Medium
Apex Trigger	Programmatic	Complex logic impossible with declarative tools	High — developer
Future / Queueable	Programmatic	Async processing, callouts, chained jobs	High — developer
Batch Apex	Programmatic	Process millions of records in manageable chunks	High — developer
Scheduled Apex	Programmatic	Scheduled complex logic at specific times	High — developer

S24 Data Management Tools

Feature	Data Loader	Data Import Wizard
Max Records	5 million	50,000
Installation	Desktop app — needs install	Web-based — no install needed
Supported Objects	All standard & custom objects	Accounts, Contacts, Leads, custom objects
Operations	Insert, Update, Upsert, Delete, Export All	Insert, Update, Upsert only
Automation	Command-line support — can schedule	Manual only — no automation
Best For	Large volumes, all object types, CI/CD	Small-medium, simple quick imports

■ **Tip:** Recycle Bin = Holds deleted records for 15 days — records can be restored with UNDELETE. ETL Tools (e.g. MuleSoft, Informatica) for complex data migration/integration.

S25 Deployment in Salesforce

Deployment = Moving metadata (config, code, customizations) from one org to another (Sandbox → Production)

Method	Description	Best For
Change Sets	Package of metadata between connected orgs. UI-based.	Admins, simple deployments
Salesforce CLI (SFDX)	Command-line source-driven development with version control.	Developers, CI/CD pipelines
ANT Migration Tool	XML-based command-line tool using package.xml	Automated pipelines
VS Code + SFDX Extension	Modern IDE-based development workflow	Modern developer workflow
AppExchange Packages	Managed/unmanaged packages for distributing apps	ISVs and app distribution

Deployment Pipeline Stage	Details
1. Developer Edition / Sandbox	Write & unit test code

Deployment Pipeline Stage	Details
2. QA / Partial Sandbox	Quality assurance testing
3. UAT / Full Sandbox	User acceptance testing with full data copy
4. Production	Deploy to live — always test thoroughly before this step

End of Notes — All Topics Covered

Git (Setup | Workflows | Core Commands | Undo | Stash | Branching | Merge vs Rebase | Conflicts | PR | Interview Q&A;) | REST API (What is API | REST Rules | HTTP Methods | Request/Response | Status Codes | Rate Limiting | Idempotency | Auth/JWT | Sync vs Async | REST vs SOAP) | Salesforce (Cloud Computing | MVC | Org Types | Objects & Fields | Relationships | Security Model | Tabs & Apps | SOQL & SOSL | DML | Formula/Rollup/Validation | Flows | Approval | Visualforce vs LWC | LWC Communication | VF Controllers | Apex | Triggers | Governor Limits | Async Apex | Database Class | Integration | Profiles/Sharing | Reports/Dashboards | Process Automation | Data Management | Deployment)